

Fine-Tuning Gemma 2-2B for Arabic Word Sense Disambiguation

Final Year Project — Implementation Report Author: Rokaya Al Harakeh Date: July 2026

1. Objective

Reproduce the Gemma result from the paper "*Zero-Shot and Fine-Tuned Evaluation of Generative LLMs for Arabic Word Sense Disambiguation*" (ArabicNLP 2025), scaled down from **Gemma 2-9B** to **Gemma 2-2B** so that the whole pipeline runs on a **free Google Colab T4 GPU**.

Task. Given an Arabic sentence, a target word, and a set of candidate **sense IDs with Arabic definitions**, predict the correct sense ID. **Dataset A** is the El-Razzaz gloss-based dataset (short sentences).

2. Methodology

The system is a four-stage pipeline, parameterised through environment variables so the same code can serve later phases (for example a Gemma 4-E4B experiment):

1. **Dataset build** — convert the El-Razzaz training split into Alpaca-format records `{instruction, input, output}`, where `output` is the gold sense ID.
2. **Fine-tuning** — LoRA (QLoRA, 4-bit) on Gemma 2-2B.
3. **Inference** — load the tuned model, generate one sense ID per target word, and extract it with a regular expression.
4. **Evaluation** — accuracy and macro-F1 against the gold labels.

Hyperparameters (kept from the paper's recipe): LoRA `r = 32`, `alpha = 32`, `dropout = 0.05` on all attention and MLP projections; 4-bit quantization; `max_seq_len = 1024`; 3 epochs; learning rate `2e-4`; `adamw_8bit` optimizer; batch size 1 with gradient accumulation 8; seed 3407.

3. Problems Faced and How They Were Solved

Most of the effort went into diagnosis. Each problem below was isolated, root-caused, and fixed; the work is captured as a sequence of focused commits on the project branch.

3.1 Lost progress on Colab disconnects

Problem. The free Colab runtime disconnects frequently, and the inference stage wrote its results only at the very end, so any disconnect discarded the entire run. **Solution.** Inference was made **resumable**: a per-sentence checkpoint file lets a re-run skip completed sentences and continue, then assemble the final predictions in the correct order.

3.2 Every prediction returned "none" (the central bug)

Problem. The fine-tuned model produced no usable sense IDs. The cause was found by testing one layer of the system at a time:

Test	Observation	Conclusion
Training vs. inference prompt	mismatched formatting	a real bug (fixed) — but not the cause
Model output	only blank lines	the model emits nothing meaningful
Tokenizer round-trip	correct	the tokenizer is fine
Base model under the framework	blank lines and a crash	the framework's forward pass is broken
Base model, plain library	fluent text	the underlying library is fine
The fine-tuned adapter	garbage output	the trained adapter itself is corrupt

Root cause. The Colab `pip install -U` command pulled a bleeding-edge software stack (**Unsloth 2026.6.6 / Transformers 5.5.0**) whose Gemma 2 forward pass was broken. Training had run *through* that broken forward pass — the tell-tale sign was an initial training loss of about **25**, which is *above* the random baseline (about 12.5) — so the resulting LoRA adapter was meaningless. The first training run was unrecoverable.

3.3 Rebuilding on a stack that works

Solution. **Unsloth was dropped entirely** (it is only a training speed-up and was never needed for inference), and both scripts were rewritten to use standard Hugging Face **QLoRA**: a 4-bit base model with a PEFT LoRA adapter, saving the **adapter** and loading base + adapter at inference time. All hyperparameters were retained.

3.4 Training library API had changed

Problem. The `trl SFTTrainer` rejected the arguments used previously — they had been moved into a separate configuration object. **Solution.** `trl` was also dropped. The text is pre-tokenised and training uses the core `transformers.Trainer` with a causal-language-modeling data collator. This reduced dependencies and improved stability.

3.5 Training was far too slow

Problem. About 17 seconds per step, plus a 12-minute evaluation pass, implied roughly a full day for three epochs on a T4. **Solution.** Attention was switched to `sdpa`, the (unused) evaluation loop was turned **off by default**, and epoch count, checkpoint frequency and evaluation batch size were made configurable so that time could be traded against fidelity.

3.6 Colab GPU usage limits and disconnects

Problem. The free tier allowed only about 1.5-3 hours of GPU per session before cutting off, with multi-hour cooldowns. **Solution.** Checkpoints are written every 100 steps (so a disconnect loses at most about 25 minutes) and training **auto-resumes from the last checkpoint**. Kaggle (12-hour uninterrupted sessions) and Colab Pro were evaluated as backup platforms.

3.7 Google Drive storage full

Problem. Drive writes began failing mid-run, threatening the final model save. **Solution.** The old, corrupt 5 GB model was deleted and the Drive Trash emptied (deleted files keep counting against the quota until the trash is emptied).

3.8 Library version clash blocking inference

Problem. The latest PEFT hard-errored on an old `torchao` version that Colab preinstalls, even though `torchao` is never used by this project. **Solution.** A small guard makes PEFT treat `torchao` as absent, so it skips that code path.

4. Final Configuration

- **Approach:** plain `transformers` + PEFT QLoRA (no Unsloth, no `tr1`).
- **Model:** Gemma 2-2B, 4-bit, LoRA `r = 32 / alpha = 32`, `sdpa` attention (matched across training and inference).
- **Resilience:** checkpoint every 100 steps with auto-resume on disconnect; resumable inference.
- **Configurable knobs (environment variables):** epochs, save/eval frequency, evaluation batch size, sequence length, and smoke-test limits.
- All changes were version-controlled, giving a clear audit trail from the initial prompt/resume fix through the final version-clash guard.

5. Results

The fine-tuned Gemma 2-2B was evaluated on the **full Dataset A test set (3,110 instances)**.

Final results (Dataset A, fine-tuned):

Accuracy (%)	Precision (macro)	Recall (macro)	Macro-F1	Instances
90.42	0.831	0.838	0.833	3,110

Comparison with the paper (Dataset A, fine-tuned):

Model	Params	Accuracy (%)	Macro-F1	Source
Gemma 2-9B	9B	89.39	81.72	paper
LLaMA 3.1-8B	8B	90.42	83.20	paper
Qwen 2.5-7B	7B	90.77	83.98	paper
Gemma 2-2B (this work)	2B	90.42	83.33	this project

The fine-tuned Gemma 2-2B reaches **90.42% accuracy and 83.33 macro-F1**, which **exceeds the paper's Gemma 2-9B** (89.39 / 81.72) and **matches the 8B-class models** (LLaMA 3.1-8B, Qwen 2.5-7B) — despite being less than a quarter of their size. This shows that, for gloss-based Arabic WSD on Dataset A, a small 2B model fine-tuned with LoRA is highly competitive with much larger models.

A pre-run validation on a 10-word slice had already indicated this (9 / 10 correct, with the single miss a near-tie between two adjacent senses), and manual probes — including a grammatical-emphasis example (Arabic *tawkid*) — returned the contextually correct sense with clean `id<eos>` output. The full-set 90.42% is consistent with that preview. (The low precision on a handful of rare sense classes — flagged by scikit-learn — is expected when many senses appear only once or twice, the same pattern the paper reports.)

6. Key Lessons

1. On a fast-moving free platform, **unpinned "latest" packages are a real risk** — a broken framework version silently corrupted an entire training run. Preferring standard, well-supported libraries (plain `transformers` / PEFT) proved far more robust than a specialised speed-up framework.
2. **A training loss above the random baseline is a red flag** worth stopping for, not a warm-up artefact.
3. **Checkpoint frequently and make every stage resumable** when the compute platform is unreliable — this turned repeated disconnects from run-killers into minor setbacks.
4. **Systematic, layer-by-layer isolation** (tokenizer, then base model, then merged model, then adapter) is what turned a vague "all outputs are wrong" into a precise root cause.